SKILL Week 9

Task1 : To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, and Test Job Listings.

Aim : To implement a simple job-management system that can create background jobs, assign job IDs, maintain their process IDs and states, list active jobs, and update job status when processes finish.

Step 1: Create the job-management program

nano job_manager.c

```
  GNU nano 7.2                          job_manager.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char state[20];
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

void create_job()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    if (pid == 0)
    {
        printf("Background job started. PID = %d\n", getpid());

        sleep(5);

        printf("Background job finished. PID = %d\n", getpid());

        exit(0);
    }
```

```
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    snprintf(jobs[job_count].state,
             sizeof(jobs[job_count].state),
             "Running");

    printf("[%d] %d Running\n",
           jobs[job_count].job_id,
           jobs[job_count].pid);

    job_count++;
}

void update_jobs()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        if (strcmp(jobs[i].state, "Running") == 0)
        {
            pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

            if (result == jobs[i].pid)
            {
                snprintf(jobs[i].state,
                         sizeof(jobs[i].state),
                         "Completed");
            }
        }
    }
}

void list_jobs()
{
    update_jobs();

    printf("\nJob ID\tPID\tState\n");
    printf("------------------------\n");
```

```c
    for (int i = 0; i < job_count; i++)
    {
        printf("[%d]\t%d\t%s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].state);
    }

    printf("\n");
}

int main()
{
    int choice;

    while (1)
    {
        printf("1. Create background job\n");
        printf("2. List jobs\n");
        printf("3. Exit\n");
        printf("Enter choice: ");

        scanf("%d", &choice);

        if (choice == 1)
        {
            create_job();
        }
        else if (choice == 2)
        {
            list_jobs();
        }
        else if (choice == 3)
        {
            printf("Exiting job manager.\n");
            break;
        }
        else
        {
            printf("Invalid choice.\n");
        }
    }

    return 0;
}
```

Step 2 — Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano job_manager.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$  gcc job_manager.c -o job_manager
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./job_manager
1. Create background job
2. List jobs
3. Exit
Enter choice: 1
[1] 551 Running
1. Create background job
2. List jobs
3. Exit
Enter choice: Background job started. PID = 551
2

Job ID  PID     State
-------------------------
[1]     551     Running

1. Create background job
2. List jobs
3. Exit
Enter choice: Background job finished. PID = 551
1
[2] 554 Running
1. Create background job
2. List jobs
3. Exit
Enter choice: Background job started. PID = 554
2

Job ID  PID     State
-------------------------
[1]     551     Completed
[2]     554     Running

1. Create background job
2. List jobs
3. Exit
Enter choice: Background job finished. PID = 554
```

Step 3: Test invalid input

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./job_manager
1. Create background job
2. List jobs
3. Exit
Enter choice: 5
Invalid choice.
1. Create background job
2. List jobs
3. Exit
Enter choice: 3
Exiting job manager.
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Observation:

The job-management system successfully creates background processes using fork(). Each background process is assigned a unique Job ID and its Process ID is stored. The job list displays the Job ID, PID, and current state of each job. The waitpid() system call with WNOHANG is used to check whether a background process has completed without blocking the parent process. The job state is updated from Running to Completed when the process finishes. Multiple background jobs and invalid menu choices were also tested successfully.

Result:

The simple job-management system was successfully implemented and tested. It can create background jobs, assign and display job IDs and PIDs, list active jobs, update process states, and handle multiple jobs correctly.

Task 2 : To Design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, and Test Foreground Switching.

Aim : To implement foreground job control by identifying a target job, transferring terminal control to the selected process, resuming a stopped process, waiting for its completion, updating the job state, and testing foreground switching.

Step 1: Create the program file

nano foreground_job.c

```
  GNU nano 7.2                                    foreground_job.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

int main()
{
    pid_t pid;
    int status;

    printf("Creating a background job...\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n", getpid());

        sleep(3);

        printf("Child process completed. PID = %d\n", getpid());

        exit(0);
    }

    printf("Job created. PID = %d\n", pid);

    sleep(1);

    printf("Transferring terminal control to the foreground job...\n");

    if (kill(pid, SIGCONT) == -1)
    {
        perror("SIGCONT failed");
    }

    printf("Waiting for foreground job to complete...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Foreground job completed successfully.\n");
        printf("Exit status: %d\n", WEXITSTATUS(status));
    }

    printf("Job state updated: Completed\n");

    return 0;
}
```

Step 2: Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano foreground_job.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc foreground_job.c -o foreground_job
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./foreground_job
Creating a background job...
Job created. PID = 607
Child process started. PID = 607
Transferring terminal control to the foreground job...
Waiting for foreground job to complete...
Child process completed. PID = 607
Foreground job completed successfully.
Exit status: 0
Job state updated: Completed
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Observation:

The foreground job-control program successfully creates a child process using fork(). The child process is identified using its Process ID (PID). The SIGCONT signal is used to resume the selected process, and waitpid() is used by the parent process to wait for the foreground job to complete. The termination status is checked using WIFEXITED() and WEXITSTATUS(). After the process finishes, the job state is updated to Completed.

Result:

Foreground job control was successfully implemented and tested. The program can identify a target job, resume the process, wait for its completion, obtain its termination status, and update the job state after completion.

Task 3: To analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior using a C program and Linux commands.

Aim : To study how signals are delivered to processes, how signal masks block and unblock signals, how process groups are identified, and how terminal signals such as SIGINT and SIGTSTP affect processes.

Step 1: Create the signal-handling program

nano signal_demo.c

```
  GNU nano 7.2                                        signal_demo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_sigint(int signal)
{
    printf("\nSIGINT received. Signal number = %d\n", signal);
}

void handle_sigtstp(int signal)
{
    printf("\nSIGTSTP received. Signal number = %d\n", signal);
}

int main()
{
    sigset_t mask;

    signal(SIGINT, handle_sigint);
    signal(SIGTSTP, handle_sigtstp);

    printf("Process ID (PID): %d\n", getpid());
    printf("Process Group ID (PGID): %d\n", getpgrp());

    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);

    printf("\nBlocking SIGINT for 10 seconds...\n");

    sigprocmask(SIG_BLOCK, &mask, NULL);

    sleep(10);

    printf("Unblocking SIGINT...\n");

    sigprocmask(SIG_UNBLOCK, &mask, NULL);

    printf("SIGINT is now unblocked.\n");
    printf("Press Ctrl+C to test SIGINT.\n");
    printf("Press Ctrl+Z to test SIGTSTP.\n");

    while (1)
    {
        sleep(1);
    }

    return 0;
}
```

Step 2: Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano signal_demo.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc signal_demo.c -o signal_demo
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./signal_demo
Process ID (PID): 662
Process Group ID (PGID): 662

Blocking SIGINT for 10 seconds...
^C^C^C
```

Task-4 : To implement resuming of stopped jobs by sending continue signals, updating job status, maintaining a job table, verifying execution, and testing background recovery.

Aim : To demonstrate how a stopped process can be resumed using SIGCONT, update its job status from STOPPED to RUNNING, and manage the job using a simple job table.

Step 1: Create the C program

nano resume_job.c

```
  GNU nano 7.2                                  resume_job.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>

struct Job
{
    int job_id;
    pid_t pid;
    char state[20];
};

int main()
{
    struct Job job;
    int status;

    job.job_id = 1;
    strcpy(job.state, "RUNNING");

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n", getpid());

        for (int i = 1; i <= 10; i++)
        {
            printf("Child is running... %d\n", i);
            sleep(1);
        }

        printf("Child process completed.\n");
        exit(0);
    }
```

GNU nano 7.2 resume_job.c *

```
    job.pid = pid;

    printf("\nJob created:\n");
    printf("Job ID: %d\n", job.job_id);
    printf("PID: %d\n", job.pid);
    printf("State: %s\n", job.state);

    sleep(2);

    printf("\nStopping the job...\n");

    kill(job.pid, SIGSTOP);

    strcpy(job.state, "STOPPED");

    printf("Job ID: %d\n", job.job_id);
    printf("PID: %d\n", job.pid);
    printf("State: %s\n", job.state);

    sleep(2);

    printf("\nSending SIGCONT to resume the job...\n");

    kill(job.pid, SIGCONT);

    strcpy(job.state, "RUNNING");

    printf("Job ID: %d\n", job.job_id);
    printf("PID: %d\n", job.pid);
    printf("State: %s\n", job.state);

    printf("\nWaiting for the job to complete...\n");

    waitpid(job.pid, &status, 0);

    if (WIFEXITED(status))
    {
        strcpy(job.state, "COMPLETED");
    }

    printf("\nFinal Job Table\n");
    printf("-------------------------\n");
    printf("Job ID\tPID\tState\n");
    printf("-------------------------\n");
    printf("[%d]\t%d\t%s\n",
           job.job_id,
           job.pid,
           job.state);

    return 0;
}
```

Step 2: Compile and run the program

```
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ nano resume_job.c
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ gcc resume_job.c -o resume_job
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$ ./resume_job

Job created:
Job ID: 1
PID: 743
State: RUNNING
Child process started. PID = 743
Child is running... 1
Child is running... 2

Stopping the job...
Job ID: 1
PID: 743
State: STOPPED

Sending SIGCONT to resume the job...
Job ID: 1
PID: 743
State: RUNNING

Waiting for the job to complete...
Child is running... 3
Child is running... 4
Child is running... 5
Child is running... 6
Child is running... 7
Child is running... 8
Child is running... 9
Child is running... 10
Child process completed.

Final Job Table
-------------------------
Job ID  PID     State
-------------------------
[1]     743     COMPLETED
alurusaranya@LAPTOP-0PA61QEN:~/OSSP$
```

Observation:

The program successfully created a child process and maintained its Job ID, Process ID, and state in a job table. The SIGSTOP signal was used to stop the child process, and its state was updated from RUNNING to STOPPED. The SIGCONT signal was then sent to the stopped process, allowing it to resume execution and changing its state back to RUNNING. The waitpid() system call was used to wait for the resumed process to complete. After completion, the final job state was updated to COMPLETED.